Group members: Ashwin Varkey 23115420
Chukwudi Williams 23061170
Manik Malik 23169717

Assignment 3

Given:

Loop length (N), Data transfer volume (V), Instructions (W), Memory bandwidth (bs) and Peak execution capability (pe), Clock frequency (f)

1. A) Execution time of loop (No overlap) =

$$\frac{f\left(\frac{cycles}{sec}\right)*V(bytes)}{bs\left(\frac{bytes}{sec}\right)*N}+\frac{W(instructions)}{Pe\left(\frac{instructions}{cycle}\right)*N}$$ [Cycles per iterations]

B) Execution time of loop (Full overlap) =

$max(\frac{f*V}{bs*N},\frac{W}{Pe*N})$ [Cycles per iterations]

C) Model for execution performance (instructions per second)

Case A: (No Overlap)

Instruction [W] / Execution time [sec] = $\frac{W}{f*\frac{V}{bs}+\frac{W}{Pe}}*f$

Case B: (With Overlap)

Instruction [W] / Execution time [sec] = $\frac{W}{f*\frac{V}{bs}}*f$ or $\frac{W}{\frac{W}{Pe}}*f=Pe*f$

(depending on whether (f*V)/bs or W/Pe is greater)

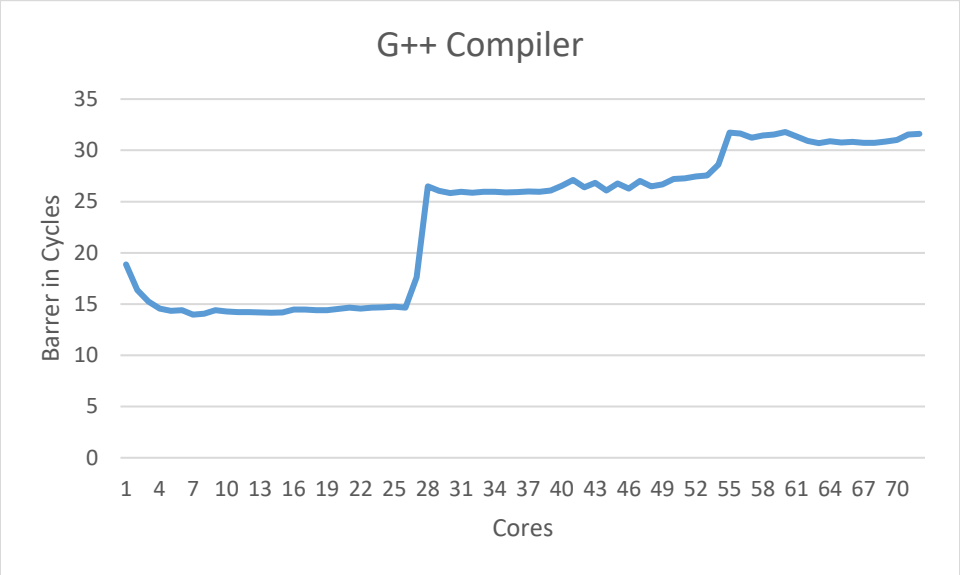
2)
a) Benchmark code:

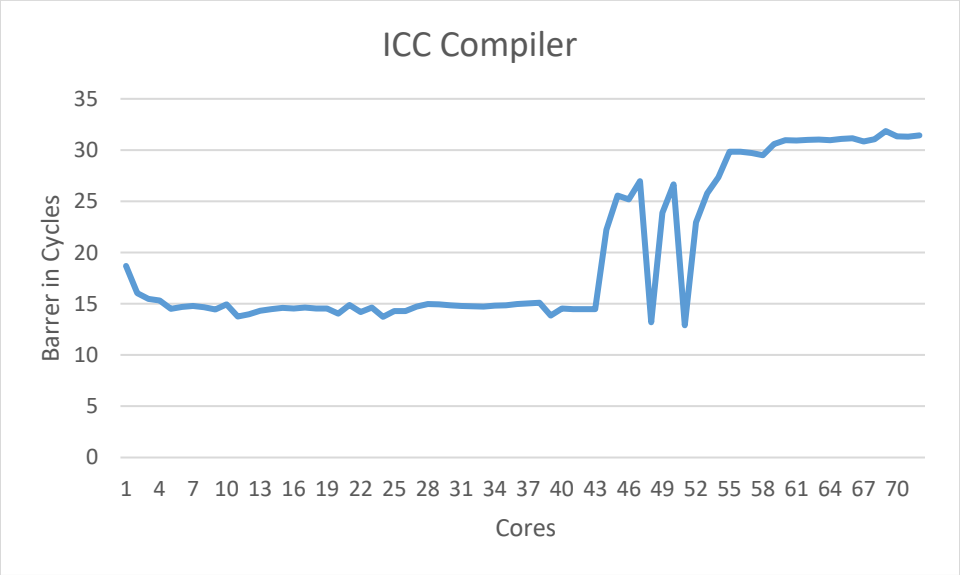
```
#include <omp.h>
#include <chrono>
#include <iostream>


using namespace std;

int nodes(double Threads, int Itr)
{
    omp_set_num_threads(Threads);
    //cout<<"Thread: "<< Threads<<'\n';
    double wcTime, wcTimeStart, wcTimeEnd, sum = 0.;
    wcTimeStart = omp_get_wtime();
    double* A = new double[Itr];
    double* B = new double[Itr];
    double* C = new double[Itr];
    for (int i = 0; i < Itr; i++)
    {
        A[i] = 3.1414;
        B[i] = 1.414;
        C[i] = 2.2360;
    }

#pragma omp parallel for
    for (int i = 0; i < Itr; i++)
    {
        A[i] = A[i] + B[i] * C[i];
        sum += A[i];
    }


    //cout << sum<<'\n';
    wcTimeEnd = omp_get_wtime();
    wcTime = wcTimeEnd - wcTimeStart;
    double freq = 2.e9;
    cout << (wcTime * freq) / Itr << '\n';
    return 0;
}

int main() {
    for (int i = 1; i <= 72; i++)
    {
        nodes(i, 1.e8);
    }
```

The above benchmark code is compiler with:
icc -O3 -fopenmp assignment8.cpp
g++ -O3 -fopenmp assignment8.cpp

and the corresponding graphs are attached below.

2b)





It is evident from the graph that initially the barrier is lowered as the problem size is big enough, but as cores are increased the synchronization time increases and after a certain number of cores, the Barrer time will increase.

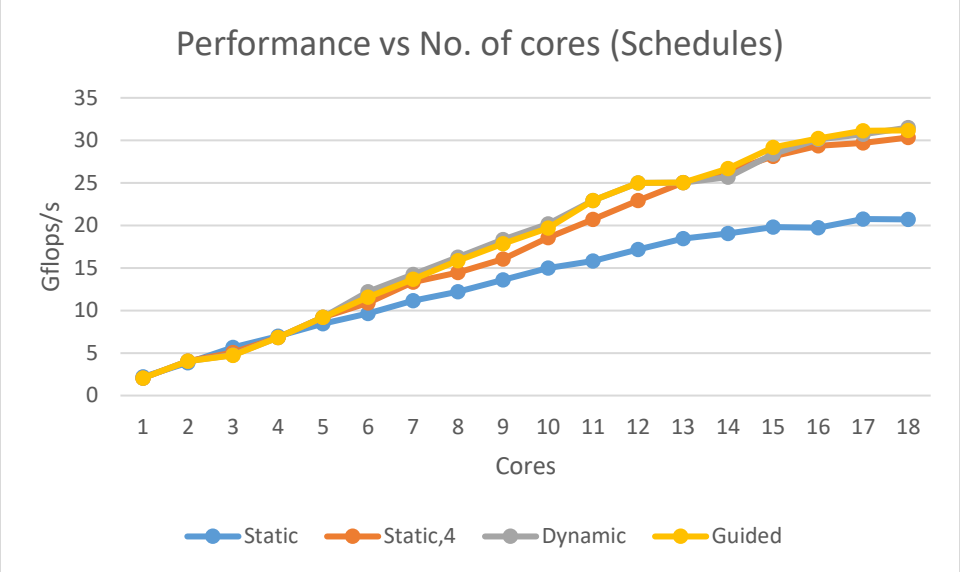
c) For the given intel Ivy Bridge the Peak performance = ncore * nFP(superscalar)*nFMA*nSIMD*f
= (10*2*1*4*2.2)= 160 GFlops/s or 80 Flops/Cycle

For Fritz we have = Peak performance = ncore * nFP(superscalar)*nFMA*nSIMD*f
Assuming f= 2GHz, = (64*2*2*4*2.2) = 2048 GFlops/s or 1024 Flops/Cycle
So Fritz can **do 12.8 times** the work than intel Ivy Bridge.

Assuming the same benchmark we applied gets a barrier of 2000 cycles that is 2000*80 = 160000 Flops that can be performed but are not for intel ivy

For fritz we are getting a barrier of around 32 cycles = 32*1024 Flops = 32768 Flops that can be performed but are lost also known as barrier pain.
So fritz is faster than ivy as per our calculations.

3) **Triangular parallel MVM**
a) The #pragma omp parallel directive creates a parallel region where multiple threads are spawned to execute the enclosed code block. The private(i, k) clause specifies that each thread should have private copies of the loop variables i and k to prevent data race conditions.



Benchmark code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>

double getTimeStamp()
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1.e-9;
}

int main() {
    int k, i, NITER;
    double  wct_start, wct_end;
    const int N = 10000;
```

```
    printf("Wall time \t Array Size \t Performace (MFlop/s)\n");
    double** a = (double**)malloc(N * sizeof(double*));
    double* x = (double*)malloc(N * sizeof(double));
    double* y = (double*)malloc(N * sizeof(double));

    for (int i = 0; i < N; ++i) {
        a[i] = (double*)malloc(N * sizeof(double));
    }

#pragma omp parallel for
    for (int r = 0; r < N; ++r) {
        for (int c = 0; c <= r; ++c) {
            a[r][c] = 1.0;  // Example value, you can modify as needed
        }
        x[r] = 1.0;  // Example value, you can modify as needed
        y[r] = 0.0;  // Initialize output vector
    }

        NITER = 1;
        do {
            // time measurement
            wct_start = getTimeStamp();
#pragma omp parallel private(c,k)
            {
                for (k = 0; k < NITER; k++) {
#pragma omp for schedule(static)
                    for (int r = 0; r < N; ++r)
                        for (c = 0; c <= r; ++c)
                            y[r] = y[r] + a[r][c] * x[c];
                    if (y[N] < 0.0) printf("%f", y[N]);
                }}
            wct_end = getTimeStamp();
            NITER = NITER * 2;
        } while (wct_end - wct_start < 1.0);
        NITER = NITER / 2;
        double time = (wct_end - wct_start) / 1e3;
        double update_mflops = (2.0 * N * N * NITER) / ((time) * 1e6);
        printf("%f\t %d\t\t %f \n", wct_end - wct_start, N, update_mflops);
        free(a);
        free(b);
        free(c);



        return 0;
}
```

b) **Within the parallel region, the #pragma omp for directive is used to parallelize the outer loop over variable r.** By parallelizing the outer loop, multiple threads can work on different iterations of the loop simultaneously, potentially improving the overall performance of the code.

```
#pragma omp parallel private(c,k)
            {
                for (k = 0; k < NITER; k++) {
#pragma omp for schedule(static)
                    for (int r = 0; r < N; ++r)
                        for (c = 0; c <= r; ++c)
                            y[r] = y[r] + a[r][c] * x[c];
                    if (y[N] < 0.0) printf("%f", y[N]);
```

Static scheduling is commonly preferred for regular tasks where each iteration of the loop takes the same amount of time. On the other hand, dynamic scheduling is better suited for irregular tasks where iterations may have varying execution times.
`The optimal Open MP loop schedule is Guided Loop Schedule in this particular case since the load balancing is improved as shown in graph below`. Guided scheduling aims to enhance global load balancing by adjusting the granularity of task chunks. It is more effective to assign smaller tasks towards the end of the parallel loop, thereby reducing differences in completion times among threads. This approach helps to evenly distribute the workload and optimize the efficiency of parallel execution.

`c)` Plot performance vs. number of cores for the default loop schedule and the best loop schedule you could identify.